

Architectural Principles for Inline Control Evidence Generation in Open-Source Data Platforms

A reference architecture and governance model for creating, operating, and publishing trustworthy data products

Chandra S Kallur

Thesis. In a regulated enterprise, an open source data platform earns its place only when the control evidence falls out of the same machinery that produces the data. If proving provenance, quality, or retention needs a second system running beside the first, the design has already failed. Portability, lineage, reproducibility, quality, identity, policy enforcement, retention, and auditability are platform concerns. They do not belong in a compliance layer bolted on afterward.

Abstract

Regulated organizations are being pushed in three directions at once. They are told to pull more economic value out of their data, to move faster on analytics and machine-learning work, and to show stronger control over provenance, privacy, operational resilience, and technology risk. Most existing platforms were not built with all three in view. Storage and compute get tuned for delivery; lineage, quality, retention, identity, and audit are added later, usually under pressure. That ordering creates a structural problem rather than a cosmetic one. When evidence has to be reassembled after an incident, an audit request, or a regulatory examination, the organization often cannot show that the data product a consumer is using is the same one that was tested, approved, and published.

What we describe here is a platform pattern in which the data product, its metadata, and its control evidence are produced together, in a single pass. The reference stack uses open storage formats (Parquet and Avro), Apache Iceberg for governed analytical tables, Spark and PySpark on Kubernetes for execution, OpenLineage for run lineage, Great Expectations for quality assertions, DataHub for metadata and governance, OpenTelemetry for operational telemetry, and Grafana for visualization, with enterprise identity federation, policy-as-code, and GitOps handling controlled change. Google Cloud Storage stands in as the example object store. The design keeps data semantics away from any single proprietary compute engine, and that is deliberate.

The argument runs against a common reading of open source as mainly a licensing decision. We treat it instead as a control, portability, and automation architecture. A platform built this way should let an organization answer, in a form a machine can check, where a data product came from, which code and which source data produced it, what quality controls ran, who approved and accessed it, how long it has to be kept, what depends on it downstream, and whether it can be rebuilt as it stood at an earlier point in time. A Data Fabric and an active-metadata control plane push this further. Metadata events, not only human tickets, drive validation, classification, orchestration, publication, quarantine, and remediation. These properties line up well with regulatory expectations around accountability, data integrity, risk-data aggregation, operational resilience, and demonstrable governance.

Keywords: open source data platform; data products; data fabric; active metadata; metadata-driven automation; data mesh; Apache Iceberg; Delta Lake; Apache Hudi; Lance; Parquet; Avro; data lineage; data quality; regulatory compliance; operational resilience; zero trust; policy as code; auditability; reproducibility; AI governance.

Contents

1. Introduction and research proposition
 2. What an open source data platform is—and is not
 3. The problems the platform should solve
 4. Regulatory and control design drivers
 5. Architectural principles
 6. Reference logical and physical architecture
 7. Data Fabric layers, active metadata automation and key architectural decisions
 8. Data products as governed deployable units
 9. Security, identity and privacy architecture
 10. Lineage, quality, metadata and observability as evidence
 11. LLM and agent integration for regulated analytics
 12. DevSecOps and software supply-chain controls
 13. Operating model and governance
 14. Reliability, resilience and disaster recovery
 15. Anti-patterns and failure modes
 16. Implementation roadmap
 17. Architecture decision checklist
 18. Conclusion
- References

1. Introduction and Research proposition

Enterprises keep describing their target state as a “data platform,” and the phrase covers a lot of ground. Some of these platforms are centralized data lakes. Some are cloud warehouses. Others are a loose collection of ingestion, orchestration, catalog, and BI products that grew up separately. For a highly regulated organization the definition has to be tighter. The platform has to process data well, but it also has to enforce, and then demonstrate, the institutional rules that govern how data may be acquired, transformed, retained, shared, explained, and retired.

The regulatory reasoning is not complicated. GDPR requires organizations that process personal data to work through purpose limitation, data minimization, accuracy, storage limitation, integrity and confidentiality, and accountability [1]. In financial services, BCBS 239 stresses accurate, complete, and timely risk-data aggregation and reporting [2]. DORA requires a documented ICT risk-management framework and real operational-resilience capability [3]. NIST zero-trust guidance drops implicit trust based on network location and moves access decisions onto users, assets, and resources [4]. The specifics vary by jurisdiction and by sector. The architectural consequence does not: control state and data state cannot be run as independent concerns.

Research proposition. An open source data platform is easiest to defend in a regulated setting when its design pushes four properties as far as they will go: portability, inspectability, reproducibility, and policy-enforced evidence. Together they lower technology-concentration risk and the recurring cost of proving compliance.

2. What an open source data platform is—and is not

An open source data platform, as we use the term, is a composable set of storage, processing, governance, security, observability, and delivery capabilities, built mostly on open standards and open-source implementations, with interfaces stable enough that a single component can be swapped without redefining the data product. It is not “self-host everything,” and it does not mean refusing managed cloud infrastructure. A regulated organization can reasonably run managed Kubernetes, managed object storage, managed databases, or a managed identity service while keeping open data formats, open metadata standards, and portable workload definitions.

The distinction that matters is between open source components and an open architecture. A platform can be full of open source tools and still be locked in operationally: metadata scattered across systems, data products tied to proprietary table semantics, deployment pipelines that only run in one place, policy expressed only through a vendor’s console. The reverse also holds. An architecture can sit on managed infrastructure and still keep data and control portable, through open formats, declarative configuration, standard APIs, exportable metadata, and pipelines that can be reproduced somewhere else.

Dimension	Open architecture expectation	Regulated-enterprise benefit
Data representation	Open file/table formats with documented specifications; product schemas and contracts are version-controlled.	Reduces conversion risk, improves preservation and enables independent verification.
Compute	Workloads are containerized/declarative and can be scheduled on standards-based orchestration.	Limits engine concentration risk and supports tested recovery/exit strategies.
Metadata	Lineage, ownership, quality, glossary and run metadata use standard or exportable representations.	Creates evidence that can survive tool replacement.
Security	Identity, policy and secrets are externalized; least privilege is enforced independently of application code.	Supports segregation of duties and consistent access control.
Operations	Telemetry uses portable protocols; deployment state is held in Git/IaC.	Improves auditability, incident analysis and recovery.
Governance	Policy is encoded in machine-enforceable controls with human accountability.	Moves compliance from periodic inspection toward continuous assurance.

3. The problems the platform should solve

A platform program should be justified by naming the enterprise problems it fixes, not by a modernization story. In practice the same short list of failures keeps turning up.

Data lineage is a challenge. Provenance is usually fragmented: business reports and AI outputs cannot be traced back reliably to source data, transformation code, and the runs that produced them. Controls are inconsistent, because quality, retention, classification, and access rules are embedded in individual pipelines or in manual process and therefore differ from team to team. Data products are ambiguous, published without clear owners, consumers, purposes, service levels, schemas, quality thresholds, or retirement criteria. Reproducibility is limited, so a historical regulatory report often cannot be rebuilt from the exact data snapshot, code version, and configuration that produced it. Audit response is slow, because evidence has to be assembled by hand from logs, tickets, repositories, catalog entries, and whatever individuals happen to remember. There is operational concentration, where a proprietary compute, metadata, or orchestration layer becomes inseparable from data semantics and daily operating procedure. Blast-radius analysis is weak: teams cannot quickly say which downstream products a failed upstream feed, a schema change, a quality breach, or a privacy incident will affect.

4. Regulatory and control design drivers

The platform should turn regulatory requirements into reusable technical control objectives. Doing it this way keeps any one regulation from being wired into the architecture, and it lets a single piece of evidence support several obligations at once. The table below gives illustrative mappings. They are not legal interpretations.

Regulatory/control concern	Architectural control objective	Representative platform evidence
Accountability and lawful processing	Every product has an owner, approved purpose, classification, permitted consumers and retention rule.	DataHub ownership/tags; product contract in Git; policy decisions; access logs.
Data minimization / purpose limitation	Only approved attributes and purposes are exposed; raw sensitive data is not the default analytical interface.	Column classification; masking/tokenization policy; governed views; contract tests.
Accuracy and completeness	Critical data elements have explicit validation rules and trendable outcomes.	GX expectations, validation results and DataHub assertions.
Traceability / provenance	Transformation lineage must connect source, job/run and published product.	OpenLineage events, DataHub graph, immutable job/run identifiers.
Operational resilience	Platform state is reproducible, failure domains are understood and recovery is tested.	GitOps/IaC history; backup and restore tests; SLO telemetry; runbooks.
Retention / evidentiary preservation	Required records and evidence cannot be deleted before retention expiry.	GCS retention policy/Bucket Lock where appropriate; lifecycle configuration; audit logs [10].
Segregation of duties	Developers cannot unilaterally bypass production policy or alter audit evidence.	RBAC; protected branches; admission policy; independent production approvals.
Third-party / supply-chain risk	Deployed artifacts are attributable, scanned and tamper-evident.	SBOM, vulnerability results, signed images/attestations using Sigstore-style controls [11].

5. Architectural principles

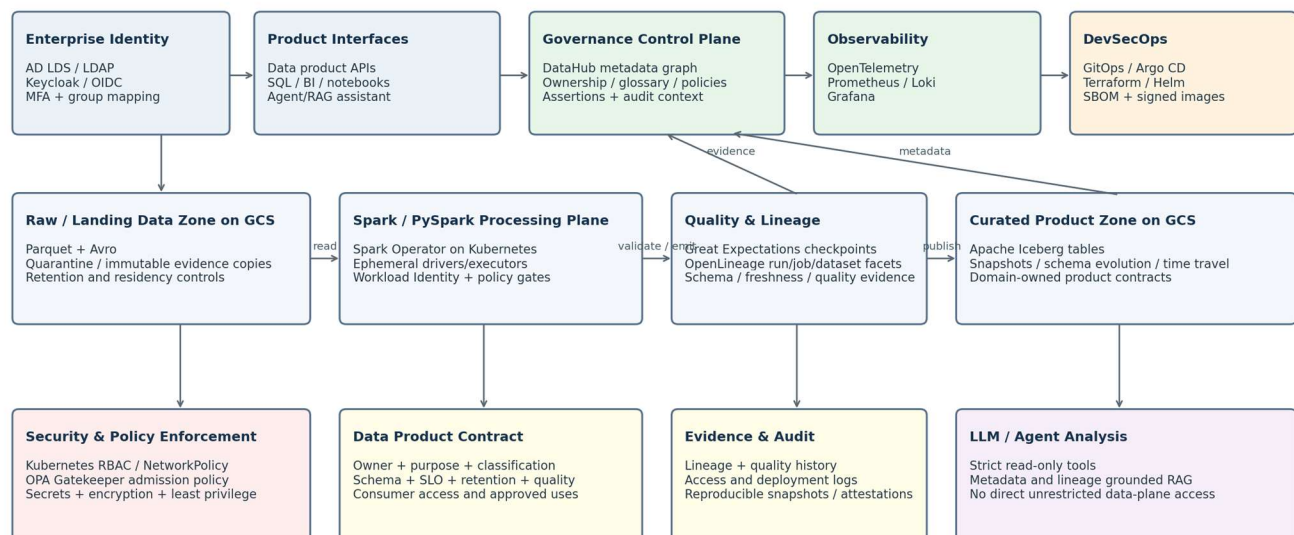
1. Evidence by design. Lineage, quality, access decisions, deployments, and observability fall out of normal execution. They are not audit artifacts assembled by hand afterward.

2. Open data, replaceable compute. Data is written in open file and table formats. Compute engines consume the data contract; they do not own it.

- 3. Metadata is a control plane.** Business context, ownership, classification, lineage, and quality status live in a graph that both people and automated policy read from.
- 4. Identity before network.** Human and workload identities are authenticated and authorized explicitly. Where a request comes from on the network is not evidence of trust, which is the zero-trust position [4].
- 5. Policy as code.** Guardrails are versioned and testable, and they are enforced at deployment and at runtime through admission control, IAM, data-access rules, and product-publishing checks.
- 6. Immutable where evidence matters.** Snapshots, retention controls, deployment history, and append-only audit trails keep evidence from being quietly rewritten.
- 7. Separation of control plane and data plane.** Governance, policy, and metadata services have to stay available to judge and constrain what the data plane does. Data processing, in turn, should not need privileged catalog credentials to run.
- 8. Domain ownership with platform standards.** Business domains own what a product means and whether it is fit for use. The platform team owns the paved roads, the shared controls, and the interoperability standards.
- 9. Automation with accountable exception handling.** Controls run automatically by default. An exception needs a stated justification, a named owner, an expiry date, and independent approval.
- 10. LLMs are consumers, not authorities.** An assistant can explain and navigate governed information. It must not become the source of truth for policy, lineage, quality, or entitlement decisions.

6. Reference logical and physical architecture

Control Plane separates policy, metadata and evidence from data plane execution



Design Objective – make control evidence a first-class platform output, not an after the fact compliance report.

Figure 1. Reference open source data platform for regulated data products.

The reference design uses Google Cloud Storage for durable object storage: Parquet and Avro in the landing and raw zones, Apache Iceberg in the curated and product zones. Iceberg helps here because its table metadata and snapshots separate the logical state of a table from how the underlying files are laid out. The specification covers schema evolution, snapshot-based table state, and atomic metadata changes [5]. That is a firmer basis for reproducibility than directory naming conventions ever were.

Spark and PySpark run transformations on Kubernetes. Kubernetes gives workload isolation and scaling; the Spark Operator gives a declarative execution model. Workload Identity, or an equivalent mechanism, should replace static cloud credentials. OpenLineage instruments Spark jobs so they emit run, job, and dataset relationships, and its facet model lets schema and other execution context ride along with the lineage event [6]. Great Expectations runs production validations through checkpoints and returns structured results [7]. DataHub pulls the resulting metadata, lineage, and quality assertions into one searchable governance graph.

OpenTelemetry is the telemetry boundary we prefer, because the Collector is vendor-neutral and can receive, process, and export traces, metrics, and logs [8]. An organization can start with Prometheus, Loki or OpenSearch, and Grafana, and still change observability backends later without re-instrumenting everything. The same logic applies to model serving: LLMs should be reached through a controlled agent or tool gateway, not wired straight into data pipelines.

7. Data Fabric layers, active metadata automation and key architectural decisions

7.1 Data Fabric as the organizing architecture

A Data Fabric is best understood as a connective layer that makes distributed data assets discoverable, governable, interoperable, and, over time, more automatable across cloud, on-premises, and domain boundaries. In a regulated enterprise it is more than a logical catalog. It is how business context, technical metadata, lineage, quality, policy, identity, operational telemetry, and data-product state get tied together, so that a control can be evaluated the same way at every stage of the lifecycle.

The platform can still use a Bronze, Silver, Gold pattern. The layers just need to be defined by control semantics rather than by color. Bronze is source-faithful, replayable evidence. Silver is standardized, conformed data. Gold is governed business products, systems of record where that applies, and consumption-ready representations. The metadata and control plane runs across all three. It is not a fourth storage zone.

Layer	Purpose in the platform	Regulated-enterprise design expectations
Source and edge	Operational systems, SaaS, files, APIs, event streams and external data feeds.	Register source ownership, system-of-record status, legal/contractual constraints, data classification and capture method before downstream use.
Ingestion / Bronze (raw)	Land source-faithful data for replay, reconciliation and evidentiary traceability.	Prefer immutable or append-only landing; preserve source timestamps and file/event identifiers; quarantine malformed data; do not silently “fix” source records.
Standardization / Silver	Clean, type, deduplicate, conform, tokenize/pseudonymize, standardize keys and apply reusable reference data.	Make transformations deterministic and versioned; emit lineage and quality evidence; distinguish business corrections from technical normalization.
Product / Gold	Publish governed domain products, regulatory datasets, analytical marts, reusable entities and approved systems of record.	Attach owner, purpose, contract, quality SLA, retention, access policy, lineage and reproducible table snapshot/version.
Semantic and access	Expose SQL, APIs, BI semantic models, governed extracts and machine/agent interfaces.	Enforce least privilege, masking/row/column controls, purpose-aware access where required, and consistent business definitions.

Layer	Purpose in the platform	Regulated-enterprise design expectations
Metadata and control plane	Connect catalog, glossary, lineage, quality, telemetry, policy, identity and deployment evidence.	Act as a cross-layer decision fabric. Metadata events should be usable by policy and automation without making the catalog itself a privileged data-processing identity.

7.2 Modeling within Silver and Gold: normalization is a logical choice, not a storage ideology

One Data Fabric design question comes up again and again: should Systems of Record in the Gold layer be modeled as highly normalized 3NF structures, or as flatter analytical ones? There is no universal answer, and the platform should not pretend there is. Normalization pays off when integrity, semantic reuse, independent entity lifecycles, and change isolation dominate. But a deeply normalized physical model can also pile up join costs and operational complexity on distributed lakehouse engines, particularly when many products keep rebuilding the same business view from the same base tables.

A workable pattern is to keep canonical, well-governed entities at the logical model level, then publish physical projections tuned for the consumption patterns that actually dominate. On Spark and Iceberg, and across multi-engine query stacks, that usually means conformed entity tables plus materialized wide or dimensional product tables wherever repeated joins would otherwise drive latency and cost. The product contract has to say which representation is authoritative, which is a derived serving projection, and how lineage connects the two. Denormalization is fine when it is controlled and reproducible. It is a problem when it is an undocumented shortcut someone took for speed.

7.3 Active metadata assisted automation

Active metadata turns the metadata layer from a passive inventory into an event-driven control system. Rather than waiting for an engineer, a steward, or an auditor to notice something, the platform treats changes in technical and business metadata as triggers for automated decisions. A new file lands, Schema drifts, Row counts move outside expected bounds, An upstream feed goes stale, a Great Expectations assertion fails, a new lineage edge appears, a classification changes, a pipeline regresses on runtime, a policy exception is about to expire. Each of these is a signal the platform can act on.

The automation loop is Sense, Contextualize, Decide, Act, Evidence. OpenLineage, Great Expectations, OpenTelemetry, storage events, and the runtime provide the signals. DataHub supplies context: ownership, criticality, classification, the dependency graph. Policy-as-code and workflow rules decide what is allowed. Spark, Kubernetes, the orchestrator, GitOps, and the access-control systems carry out the action. The outcome is written back as metadata and control evidence. For a regulated organization this loop matters for a specific reason: the same mechanism that speeds up delivery also leaves a traceable record of why an automated action was taken.

Metadata/event	Automated response	Evidence retained
New raw object or partition	Register asset/run, validate naming/schema, start approved ingestion pipeline or quarantine.	Object ID/version, source timestamp, schema fingerprint, pipeline/run ID and policy decision.
Schema change	Compare with contract; auto-accept compatible changes, route breaking changes for approval, block publication if required.	Old/new schemas, compatibility result, approver/exception and affected lineage.
Quality failure	Stop or mark publication, quarantine affected snapshot, notify owner, identify downstream blast radius.	GX validation result, threshold, dataset snapshot, incident/ticket and downstream lineage.
Freshness or late-arrival event	Delay dependent jobs, switch status to degraded, trigger backfill/reconciliation when source arrives.	Expected/observed freshness, dependency state, rerun ID and product status transition.
Classification or sensitivity change	Re-evaluate access, masking, retention and export policies before next publication.	Classification change, policy version, access decisions and impacted consumers.

Metadata/event	Automated response	Evidence retained
Runtime/telemetry anomaly	Scale, retry, route for investigation or suppress publication when technical failure can affect correctness.	Trace/span IDs, resource metrics, retries, run state and remediation action.

Enterprise Banking Case Study: From Ingestion to Audit

Take a daily exposure or liquidity data product. Source extracts and CDC events land in the raw zone and produce metadata events. Before the Spark transformation starts, the automation layer checks source registration, the expected arrival window, schema compatibility, and record-volume controls. During the run, OpenLineage records the source, job, and output relationships, OpenTelemetry captures runtime evidence, and Great Expectations evaluates completeness, uniqueness, referential integrity, and business-rule expectations. If the mandatory controls pass, the resulting Iceberg snapshot is registered as the new published version and DataHub records the lineage, quality state, and ownership context. If a critical expectation fails, or an upstream feed is stale, publication is held or the product is marked degraded, the affected downstream regulatory products are found through lineage, and an incident and remediation workflow starts. The evidence bundle at the end links the source object and version, the code commit, the Spark run, the quality result, the lineage event, the table snapshot, and the publication decision.

This is not the same thing as conventional orchestration. An orchestrator answers one question: what task runs next? Active-metadata automation answers several more. Should it run at all? Under what controls? What changed since the last run? Who is affected? And may the resulting product be published?

7.4 Storage format strategy: separate raw file format from processed table format

A regulated platform makes two separate decisions here: the file format that preserves source data, and the table format that manages processed analytical state. Collapsing them into one decision is how avoidable lock-in gets in. Parquet and Avro are file and serialization formats. Iceberg, Delta Lake, and Hudi are table formats; they manage collections of underlying columnar files plus transactional metadata. Lance sits across both categories, aimed at AI-oriented workloads.

For raw storage, source fidelity and replayability matter more than query speed. Keep the source-native representation whenever it is material to reconstructing evidence, while still standardizing the common landing patterns. For high-volume structured batch extracts, Parquet is usually the right raw analytical format, because a columnar layout makes downstream scans cheap. For events, messages, and CDC-style records, where schema-bearing row serialization and schema resolution are the point, Avro tends to fit better [15][16]. CSV and JSON should generally survive only as the contractual or native source format, or where exact replay demands them. They should not drift into being the default internal analytical representation.

Raw format	Recommended use	Control / engineering considerations
Parquet	Bulk structured/semi-structured extracts that will be scanned analytically; large batch landing from databases or applications.	Columnar and efficient for selective analytical reads; preserve source identifiers and checksums because Parquet conversion is itself a transformation [15].
Avro	Event/CDC feeds, message-oriented ingestion and records where writer schema and schema resolution are central.	Compact binary serialization with explicit schemas; well suited to evolving event contracts and replay [16].
CSV / JSON	External or legacy interchange when the source contract requires it.	Retain immutable original where evidentiary fidelity matters, but convert to typed Parquet/Avro for governed processing; record the conversion lineage.

7.5 Processed table-format decision framework

The processed-data table format should come out of real engine and workload requirements, not a generic “best format” ranking. These ecosystems are converging. Even so, interoperability, write behavior, indexing, maintenance

semantics, and engine support still differ enough that a bad early choice turns into real migration work later. The decision belongs in an Architecture Decision Record, and it should be revisited whenever the compute mix changes.

Format	Choose when	Architectural strengths	Important qualifications for a regulated open platform
Apache Iceberg	The stack includes multiple engines; partition strategy may evolve; schema evolution is frequent; vendor-neutral catalog/API interoperability is a priority.	Broad engine integrations including Spark and Trino; hidden partitioning, partition evolution, schema evolution, snapshots/time travel and a standardized REST catalog protocol [17][18].	Recommended default for this reference architecture because the target is a multi-engine, vendor-neutral data plane. Validate feature compatibility across every engine used, not only table-format specification support.
Delta Lake	Spark/Databricks is the dominant compute environment; operational simplicity and Spark-centric optimization are more important than heterogeneous-engine neutrality.	Liquid clustering can evolve clustering choices; deletion vectors reduce some rewrite costs for row-level modifications; OSS Delta provides auto-compaction capabilities [19][20][21].	Strong choice for Spark-centric estates. Some advanced feature/protocol combinations can reduce compatibility with older or non-supporting clients; test interoperability before enabling them.
Apache Hudi	CDC, frequent upserts/deletes and streaming ingestion dominate; record-level location/indexing and low-latency incremental processing are central.	Indexing maps record keys to file locations, enabling efficient upsert/delete patterns; Merge-on-Read and record-level indexing target high-change workloads [22][23].	Prefer where write/update behavior is the primary requirement. Account for compaction, indexing and operational tuning; do not select solely because the platform has “streaming” somewhere.
Lance	Multimodal AI, vector/full-text search, feature engineering, model training and fast random access are primary workloads.	AI-oriented lakehouse format with vector/full-text search, random access, versioning/ACID semantics and multimodal support [24].	Treat as a specialized AI data plane unless enterprise analytical interoperability requirements justify broader adoption. Keep metadata, identity and lineage integration aligned with the main governed platform.

For the architecture in this paper, Spark and PySpark on Kubernetes, GCS object storage, Trino-style SQL access, likely use by other engines, and with the goal of vendor neutrality, the recommended default is Apache Iceberg. That is not a claim that Iceberg wins everywhere. A Spark or Databricks-only domain can rationally standardize on Delta Lake. A CDC-heavy operational lake can rationally pick Hudi. A multimodal AI subsystem can rationally run on Lance. Key design criteria must be to name the default enterprise standard and then define governed exception patterns, rather than forcing every workload into one format.

7.6 Key architectural decisions

The decisions below turn the preceding principles into platform standards that can actually be enforced. Each should be recorded as an ADR, with rationale, owner, compatibility assumptions, and review triggers. Format decisions in particular should capture the current engine matrix, the data-change pattern, the latency objective, the expected schema and partition evolution, and the exit strategy.

Decision	Recommended position	Why it matters in regulated environments
Raw formats	Preserve source-native files where replay/evidence requires it; use Parquet as the default for bulk analytical extracts and Avro for event/CDC records with schema-driven serialization.	Separates evidentiary source preservation from optimized processing, while keeping raw data portable and independently readable [15][16].

Decision	Recommended position	Why it matters in regulated environments
Curated table format	Default to Apache Iceberg for the shared multi-engine curated/product plane; allow governed Delta/Hudi/Lance exceptions when workload criteria justify them.	Matches table format to engine diversity and mutation/access patterns instead of assuming one format is universally optimal [17]-[24].
Object-store layout	Separate landing/quarantine, raw/evidence, curated, product and audit zones with distinct IAM/retention policies.	Reduces accidental privilege inheritance and permits different retention regimes.
Compute orchestration	Run Spark in Kubernetes with ephemeral job-scoped identities; keep executor access narrower than platform-admin access.	Limits blast radius and supports workload-level audit.
Catalog	Use an Iceberg REST/Nessie-compatible catalog pattern; keep catalog authorization and data-storage authorization independently reviewable.	Avoids making the compute engine the authoritative catalog.
Metadata governance	Use DataHub as the metadata graph, but export/snapshot critical metadata and avoid storing legal evidence only in a mutable UI.	Catalogs improve control, but should not become a new opaque system of record.
Lineage	Instrument at execution time with OpenLineage; normalize identifiers across jobs, datasets and environments.	Run-level evidence is more defensible than manually curated lineage alone.
Data quality	Treat quality rules as versioned product requirements, execute automatically, and publish results as assertions.	Creates traceable proof of what was tested and when.
Observability	Standardize telemetry on OpenTelemetry and route via collectors.	Decouples instrumentation from monitoring vendor and centralizes filtering/security [8].
Identity	Federate enterprise AD LDS through an OIDC broker such as Keycloak; distinguish human, service and workload identities.	Enables consistent authentication while preserving legacy directory integration.
Policy	Use cloud IAM + Kubernetes RBAC/NetworkPolicy + OPA Gatekeeper + data-access policy.	Controls should be layered and independently enforceable; Gatekeeper supports admission and audit policy [9].
Change management	GitOps for platform configuration; immutable artifact digests; signed release artifacts; separation of build and deploy permissions.	Creates a reviewable chain from source code to production runtime.
LLM integration	Use a strict agent tool gateway; do not provide broad direct credentials to DataHub, Trino or object storage.	Reduces prompt-injection blast radius and makes AI actions auditable.

8. Data products as governed deployable units

The data product is the unit where decentralized ownership and centralized control meet. A product should become publishable only once its contract is complete and machine-checkable. That single rule turns governance from a meeting into a deployment condition.

- Identity: globally unique product ID, domain, owner, technical steward and accountable business owner.
- Purpose: documented business purpose, approved use cases and prohibited/conditional uses.
- Classification: confidentiality, privacy category, criticality, regulatory relevance and residency constraints.
- Interface: schema, semantic definitions, keys, partitioning, update mode and compatibility expectations.
- Quality: critical data elements, expectations, thresholds, freshness/completeness rules and response to failure.
- Service level: availability, publishing cadence, recovery objectives, incident severity and support model.

- Retention: source and derived-data retention, legal hold behavior, deletion/expiry workflow and evidence retention.
- Lineage: required upstream and downstream lineage depth plus transformation/run identifiers.
- Access: approved roles/groups, row/column restrictions, masking rules and privileged-use process.
- Lifecycle: version, deprecation notice, successor product, archival conditions and destruction evidence.

A product-publishing controller can read this contract, check the latest quality results, confirm that lineage was captured, verify the required tags and owners in DataHub, validate image signatures, and confirm deployment-policy compliance. For any control marked mandatory, publication fails closed.

9. Security, identity and privacy architecture

Security should be organized around identities and resources, not trusted network zones. NIST SP 800-207 frames zero trust exactly this way: remove implicit trust based on network location, and require authentication and authorization for access to enterprise resources [4]. So the platform keeps separate identities for users, CI/CD systems, platform services, Spark drivers and executors, and agent tools. A shared, long-lived service credential is a design defect, and should be treated as one.

For the enterprise-directory requirement, AD LDS can stay an authoritative directory while Keycloak, or another OIDC/SAML broker, handles modern federation to the platform UIs and APIs. Group membership can map to platform roles. Authorization should then layer attributes on top: domain, classification, environment, purpose. Holding a “Finance Analyst” role should not, by itself, imply access to every finance dataset.

Privacy-by-design means data minimization, purpose limitation, restricted access, and technical safeguards from the start rather than as a later addition [1][12]. The platform patterns we recommend include tokenization or pseudonymization before broad analytical use, curated views that drop attributes nobody needs, column-level classification, a separate access path for re-identification, encryption in transit and at rest, time-bounded privilege, and retention enforcement tied to the product’s stated purpose. GCS retention policies and Bucket Lock can give immutable storage behavior for selected records and audit artifacts. Locking is irreversible, though, so it has to be governed as a legal and compliance decision, not set as a convenience [10].

10. Lineage, quality, metadata and observability as evidence

The strongest idea in this architecture is a simple one: make control evidence correlate through shared identifiers. A pipeline run gets one run ID, and that ID shows up in Spark execution metadata, OpenLineage events, OpenTelemetry traces, quality results, and the DataHub graph. The output Iceberg snapshot ID and the code or artifact digest are recorded next to it. What you get is an auditable chain from source event to published product.

Evidence type	What should be captured	Primary purpose
Lineage	Source and target dataset IDs; job ID; run ID; field lineage where necessary; schema facets; timestamps.	Provenance, impact analysis, incident investigation, reproducibility.
Quality	Expectation version; dataset/snapshot; observed metrics; pass/fail; exception; checkpoint/run ID.	Demonstrate fitness for use and identify systemic deterioration.
Operational telemetry	Trace/span IDs; job duration; executor metrics; failures; retries; resource consumption.	Reliability engineering and linkage between data defects and platform incidents.
Access	Human/workload identity; resource; action; policy decision; purpose/context where applicable.	Security investigation and accountability.

Deployment	Commit SHA; image digest; SBOM; signature/attestation; approval; deployment timestamp.	Software supply-chain assurance and change traceability.
Data state	Iceberg snapshot IDs, schemas, manifests, relevant GCS generations/retention state.	Reproduce the exact state used in a historical decision/report.

OpenLineage suits this because its core model separates dataset, job, and run entities and supports extensible facets [6]. Great Expectations checkpoints are built as a production validation abstraction and return results that can trigger downstream actions [7]. OpenTelemetry collectors can centralize batching, retries, encryption, and filtering, including stripping sensitive data before logs leave the workload boundary [8]. The value comes from connecting these through correlation identifiers, not from running them as three separate tools.

11. Agent integration

LLM integration must start with explanation and navigation, not autonomous data manipulation. The pattern we recommend is an Agent/RAG service that retrieves authoritative metadata, lineage, quality assertions, and approved documentation, then composes an explanation. RAG here is not just vector search. A GraphQL query against DataHub and a bounded lineage traversal are structured retrieval operations.

A strict tool gateway should expose a small set of read-only functions: `get_dataset_profile`, `get_lineage`, `get_quality_summary`, `get_recent_pipeline_runs`, `approved_query_template`. The model never sees raw cloud credentials, and it cannot invent new endpoints or arbitrary SQL. Authorization is decided outside the model, from the user identity and the dataset policy. Tool parameters are clamped, for example a maximum lineage depth and a bounded time window. Every invocation is logged, and the final explanation should name the evidence it leaned on.

Take the question “Why did the regulatory exposure product fail today?” It can trigger retrieval of the current product metadata, the upstream lineage, the last Spark run, the latest Great Expectations assertions, and recent telemetry. The model can then explain that an upstream feed was stale and that the matching completeness expectation failed, and separately list the downstream products at risk. It should not assert causality unless the evidence supports it. That line, between an evidence-backed explanation and a model inference, is where regulated use cases succeed or fail.

- Never use the LLM as the system of record for lineage, ownership, quality status, retention or entitlement.
- Do not allow free-form SQL by default; prefer pre-approved query templates and bounded parameters.
- Filter prompts and retrieved context for secrets and unnecessary personal data before model invocation.
- Version prompts, tool schemas and model endpoints; log model/tool versions used for material decisions.
- Apply human review for outputs used in regulatory reporting, customer decisions, risk acceptance or control attestation.
- Make model-provider substitution possible through a gateway so data residency and provider-risk requirements can be enforced.

12. DevSecOps and Supply-Chain controls

Open source removes some vendor lock-in and hands back a different obligation. The enterprise now owns dependency governance, patching, component provenance, and secure integration. This is the key aspect which is often overlooked both from technology as well as effort point of view. What the platform needs is a software supply-chain program, not just a container registry.

1. All application and platform configuration is version-controlled; production changes are made through protected branches and reviewed pull requests.
2. CI generates SBOMs, performs dependency and container vulnerability scanning, executes unit/integration/policy tests and records immutable build provenance.

- Images are referenced by digest, signed, and verified at admission. Sigstore/Cosign is an open-source pattern for signing and verifying software artifacts and recording signing evidence [11].
- OPA Gatekeeper (or equivalent) denies unapproved registries, privileged containers, missing resource limits, unsigned images where enforced, and prohibited host access. Gatekeeper also supports audit of existing non-compliant resources [9].
- Terraform and Helm define infrastructure and platform services; Argo CD/Flux continuously reconciles declared state and makes drift visible.
- Secrets are retrieved at runtime from a secret store; secrets are never embedded in images, Helm values committed to Git, notebook files or Spark configuration logs.
- Open-source components have explicit owners, supported-version policies, CVE response SLAs and end-of-life procedures.

13. Operating model and governance

Architecture on its own does not produce regulatory confidence. The operating model has to allocate accountability clearly across the platform team, the domain data-product teams, security, privacy, risk and compliance, and technology operations.

Role	Accountability
Enterprise Data Governance	Defines classification, ownership, critical-data, metadata and product-governance policies; approves enterprise standards.
Platform Engineering	Builds and operates paved-road capabilities, templates, runtime controls, metadata integrations and observability.
Domain / Data Product Owner	Accountable for meaning, purpose, quality thresholds, consumer commitments, retention and lifecycle of the product.
Data Engineering	Implements pipelines and product contracts; remediates quality/lineage failures; maintains technical documentation.
Security Engineering	Defines identity, workload isolation, vulnerability, secrets, network and policy-as-code standards.
Privacy / Legal	Interprets privacy obligations, lawful purpose, retention/deletion, residency and high-risk processing requirements.
Risk / Compliance / Internal Audit	Challenges control design, reviews evidence, validates exceptions and tests whether controls operate effectively.
SRE / Operations	Owns platform SLOs, incident response, backup/restore testing and resilience evidence.

A regulated platform should run an architecture-decision-record process. Decisions with regulatory impact, retention lock, cross-region replication, encryption-key ownership, identity federation, lineage retention, choice of AI model provider, exception behavior, should each carry a documented rationale, a risk assessment, a named decision owner, a review date, and an honest assessment of how reversible the decision is.

14. Reliability, resilience and disaster recovery

Resilience has to cover three kinds of state: data state, metadata and control state, and deployment state. Recovering the GCS data while losing lineage, product contracts, catalog metadata, or quality history may not be enough for a regulated process. DORA is explicit about a documented ICT risk-management framework and operational-resilience capability for covered financial entities [3], and the lesson carries well beyond the entities it formally covers.

- Define RTO/RPO separately for raw evidence, curated Iceberg tables, catalog metadata, DataHub state, quality results, observability data, identity dependencies and GitOps control repositories.

- Back up metadata databases and test restore; do not assume that object-store durability protects catalog/control-plane state.
- Test reconstruction of an Iceberg table and a material data product from a known snapshot, code digest and pipeline configuration.
- Exercise loss of a Kubernetes cluster and redeploy through IaC/GitOps into a clean environment.
- Use multi-zone designs for critical services; consider region-level recovery for obligations that justify it, while respecting data-residency restrictions.
- Keep an offline/independently accessible copy of critical runbooks, recovery credentials and dependency inventories.
- Include third-party and open-source dependency outages in resilience scenarios, not only infrastructure failures.

15. Anti-patterns and failure modes

“Open source” as a procurement label. Buying a pile of OSS products without first defining stable data contracts and interfaces can produce more integration lock-in, not less.

Catalog as decoration. If ownership, classification, and lineage are optional fields, the catalog is a search UI, not a control plane.

Manual lineage. Diagram-based lineage that is not wired to execution runs goes stale fast, and it cannot support reliable incident analysis when it is needed.

Quality after publish. Running data quality only in downstream reporting finds defects after consumers have already acted on them.

A single shared Spark identity. One shared service account makes least privilege and forensic attribution very hard.

Observability as unrestricted logging. Raw payloads, SQL text, and personal data leak into logs and traces easily. Telemetry needs its own classification, filtering, and retention controls.

Immutable everything. Overusing retention locks collides with deletion obligations and creates operational and legal risk. Immutability should be applied selectively and governed.

Uncontrolled LLM access. Give an agent unrestricted SQL or object-store credentials and prompt injection becomes a data-access path.

No open-source lifecycle policy. Unpatched or abandoned dependencies can open a control gap wide enough to matter, even when the rest of the architecture is sound.

Assuming cloud compliance equals workload compliance. A cloud provider’s certifications do not show that your organization configured access, retention, data quality, lineage, or change controls correctly.

16. Recommended Implementation roadmap

A practical adoption sequence puts evidence and standards in early, instead of deferring governance until after the platform has scaled and the habits have set.

Phase	Focus	Exit criteria
0. Control baseline	Regulatory/control inventory, data classification, threat model, technology decision records, target SLOs.	Named control owners; approved target principles; initial data-product contract.
1. Foundation	GCS zones, Kubernetes, workload identity, Spark Operator, GitOps, secrets, OTel, baseline IAM.	One reproducible environment built from IaC; no static cloud keys in workloads.

2. Governed pipeline	Parquet/Avro landing, Iceberg curated table, OpenLineage, GX, DataHub ingestion.	One end-to-end product with source→run→snapshot lineage and quality evidence.
3. Product publishing	Contract validation, publishing gate, policy-as-code, consumer access workflows, Grafana SLO dashboards.	Non-compliant product cannot be published without explicit exception.
4. Resilience & audit	Backup/restore, historical reconstruction, control evidence export, incident and DR exercises.	Demonstrated recovery and reproducibility for a material product.
5. AI-assisted analysis	Strict Agent/RAG gateway for metadata/lineage/quality explanation; model gateway and audit.	AI access bounded by existing data authorization; all tool actions attributable and logged.
6. Scale & federation	Domain onboarding templates, chargeback/cost controls, policy libraries, component lifecycle management.	Multiple domains onboard through a standard paved road with measurable control coverage.

17. Architecture decision checklist

- Does every published product have a named accountable owner, purpose, classification, retention rule and consumer contract?
- Can a material data product be reproduced from a historical snapshot using the exact code/configuration that produced it?
- Can the organization trace a product to source data and identify downstream impact without manual reconstruction?
- Are quality checks versioned, automatically executed and linked to the exact product/run/snapshot they validate?
- Are human, service and workload identities distinct, short-lived where possible and governed by least privilege?
- Can policy deny a deployment or publication before non-compliant state reaches production?
- Is telemetry filtered so logs and traces do not become an uncontrolled sensitive-data store?
- Are retention locks and legal holds independently governed, documented and reversible only where law/policy permits?
- Are all production artifacts attributable to source commits, scanned, signed/attested and deployed by controlled automation?
- Can DataHub/lineage/quality evidence be exported or preserved independently of a single metadata application?
- Are component versions, CVEs, maintenance ownership and end-of-life dates managed as part of platform risk?
- Does the LLM/agent layer use explicit tools and external authorization rather than broad credentials or prompt-only guardrails?
- Are DR tests performed for data plane, metadata/control plane and identity dependencies?
- Is the platform able to move or substitute compute/observability/model components without rewriting the data product semantics?
- Is the raw-data format decision distinct from the processed table-format decision, with source fidelity and replay requirements explicitly documented?
- Does each table-format ADR reflect the actual engine matrix, update/CDC pattern, latency target, schema/partition evolution needs and interoperability tests?
- Can metadata events automatically trigger validation, quarantine, publication gating, impact analysis or policy reevaluation without bypassing approval requirements?
- Are canonical normalized entities and denormalized serving projections explicitly distinguished and linked by lineage, rather than allowing uncontrolled duplication?

18. Conclusion

For a regulated organization, the value of an open source data platform is not that it removes commercial software and reduces license costs. The value is that it can make the architecture independently inspectable, the data portable, the execution reproducible, and the control evidence durable. That shifts the compliance conversation. The question stops being “which system says the data is governed?” and becomes “can we show, from correlated evidence, how this product was produced and controlled?”. However, it also puts a huge burden and an obligation on the organization going this path. While commercial technologies come with a strong vendor support in case new vulnerabilities surface or to establish proven integration patterns or to fix any performance issues, with so many components in the Open Source Data Platform at play, many of which could lack strong vendor backing, the onus of ensuring compliance rests with the organization.

The architecture here separates durable data state from replaceable compute on purpose. Iceberg snapshots preserve table history. OpenLineage captures execution lineage. Great Expectations validates the data. DataHub holds the governance context in one place. OpenTelemetry standardizes operational telemetry. Identity and policy are pushed into Kubernetes and the deployment workflow. LLM capability is added only through a bounded tool layer that sits on governed metadata and evidence.

The platform is best read as a control system for data products that runs continuously. The Data Fabric supplies cross-layer context. Active metadata supplies the feedback loop. Open file and table formats keep things portable. Automated evidence generation makes governance something you can observe rather than something you assert. Done well, it delivers faster product delivery and stronger regulatory assurance together, not as a trade: product teams get a paved road, and risk and compliance functions get consistent, machine-generated evidence. It can also change as workloads change, keeping Iceberg as the multi-engine default while allowing controlled Delta, Hudi, or Lance patterns where a workload genuinely fits them better. That combination, not any single open source tool, is what makes an open source data platform enterprise-grade.

References

- [1] **European Commission / EDPB.** GDPR principles of personal-data processing; accountability and privacy-by-design concepts. https://commission.europa.eu/law/law-topic/data-protection/information-business-and-organisations/principles-gdpr_en
- [2] **Basel Committee on Banking Supervision.** Principles for effective risk data aggregation and risk reporting (BCBS 239), 2013; status current. <https://www.bis.org/publ/bcbs239.htm>
- [3] **European Union.** Regulation (EU) 2022/2554, Digital Operational Resilience Act (DORA). <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32022R2554>
- [4] **NIST.** SP 800-207: Zero Trust Architecture, 2020. <https://csrc.nist.gov/pubs/sp/800/207/final>
- [5] **Apache Software Foundation.** Apache Iceberg documentation and table-format specification: snapshots, schema evolution and atomic table-state changes. <https://iceberg.apache.org/spec/>
- [6] **OpenLineage.** OpenLineage specification and facets: dataset, job and run metadata; extensible lineage event model. <https://openlineage.io/docs/spec/facets/>
- [7] **Great Expectations.** Checkpoint and Validation Result documentation; production validation and actions. https://docs.greatexpectations.io/docs/reference/api/checkpoint_class/
- [8] **OpenTelemetry.** OpenTelemetry Collector: vendor-neutral collection, processing and export of telemetry. <https://opentelemetry.io/docs/collector/>
- [9] **Open Policy Agent / Gatekeeper.** Gatekeeper policy enforcement and audit for Kubernetes. <https://open-policy-agent.github.io/gatekeeper/website/docs/>
- [10] **Google Cloud.** Cloud Storage Bucket Lock and retention policies for immutable-retention use cases. <https://docs.cloud.google.com/storage/docs/bucket-lock>
- [11] **Sigstore.** Open-source software supply-chain signing and verification tooling. <https://docs.sigstore.dev/>
- [12] **European Commission.** Data protection by design and by default. https://commission.europa.eu/law/law-topic/data-protection/information-business-and-organisations/obligations_en
- [13] **NIST.** Cybersecurity Framework (CSF) 2.0, 2024. <https://www.nist.gov/publications/nist-cybersecurity-framework-csf-20>
- [14] **Basel Committee on Banking Supervision.** Implementation of BCBS 239 Principles: supervisory and industry themes, January 2026. https://www.bis.org/publ/bcbs_nl36.htm
- [15] **Apache Software Foundation.** Apache Parquet file format documentation: column chunks, row groups and analytical columnar layout. <https://parquet.apache.org/docs/file-format/>
- [16] **Apache Software Foundation.** Apache Avro documentation and specification: schema-based binary serialization, container files and schema resolution. <https://avro.apache.org/docs/>
- [17] **Apache Software Foundation.** Apache Iceberg documentation: multi-engine integrations, schema evolution, hidden partitioning, partition evolution and time travel. <https://iceberg.apache.org/docs/latest/>
- [18] **Apache Software Foundation.** Apache Iceberg REST Catalog specification: common REST protocol for language and engine compatibility. <https://iceberg.apache.org/rest-catalog-spec/>

- [19] Delta Lake. Liquid clustering documentation. <https://docs.delta.io/delta-clustering/>
- [20] Delta Lake. Deletion vectors documentation. <https://docs.delta.io/delta-deletion-vectors/>
- [21] Delta Lake. Open-source optimizations and auto compaction. <https://docs.delta.io/optimizations-oss/>
- [22] Apache Hudi. Indexing and record-level index documentation.
https://hudi.apache.org/docs/metadata_indexing/
- [23] Apache Hudi. Streaming data lake design: continuous upserts and Merge-on-Read behavior.
<https://hudi.apache.org/blog/2026/07/21/what-is-a-streaming-data-lake/>
- [24] Lance. Open lakehouse format for multimodal AI: vector/full-text search, random access, feature engineering, ACID/time travel. <https://lance.org/>

Appendix A. Reference technology stack

Capability	Reference technology	Architectural role
Object storage	Google Cloud Storage	Durable raw, curated, product and audit storage; lifecycle, residency and retention controls.
File formats	Parquet / Avro (raw defaults; preserve source-native where required)	Portable raw/source representations: Parquet for analytical batch landing; Avro for event/CDC and schema-driven records; retain native source files where evidentiary replay requires them.
Table format	Apache Iceberg (enterprise default); governed Delta / Hudi / Lance exceptions	Iceberg for the shared multi-engine analytical plane; Delta for Spark-centric domains, Hudi for high-change CDC/upsert workloads, and Lance for multimodal AI where justified.
Processing	Apache Spark / PySpark	Batch/large-scale processing and table maintenance.
Container orchestration	Kubernetes + Spark Operator	Elastic, declarative and isolated execution of data workloads.
Lineage	OpenLineage	Execution-time job/run/dataset lineage and extensible facets.
Quality	Great Expectations	Versioned expectations, checkpoints and structured validation results.
Metadata/governance	DataHub OSS	Metadata graph, discovery, ownership, tags, glossary, lineage and assertions; supplies context for active-metadata policy and automation decisions.
Telemetry	OpenTelemetry	Vendor-neutral traces/metrics/logs; correlation across services and data runs.
Monitoring	Prometheus + Loki/OpenSearch + Grafana	Metrics/log storage and operational/data-product dashboards.
Identity	AD LDS + Keycloak	Enterprise directory federation and OIDC authentication for platform services.
Policy	Cloud IAM + K8s RBAC/NetworkPolicy + OPA Gatekeeper	Layered access, workload and deployment controls.
Secrets	Vault / External Secrets pattern	Centralized secret material and runtime retrieval.
DevOps	Git + CI + Terraform + Helm + Argo CD/Flux	Version-controlled build, infrastructure and GitOps deployment.
Supply chain	SBOM + scanning + Sigstore/Cosign pattern	Artifact provenance, integrity and vulnerability governance.
LLM analytics	vLLM/TGI/Ollama + strict Agent/RAG tool gateway	Governed AI analysis over metadata, lineage, quality and approved query interfaces.

Appendix B. Minimum evidence bundle for a critical data-product release

Product contract version and approval record.

Source-code commit SHA and pipeline configuration digest.

Container image digest, SBOM, vulnerability results and signature/attestation.

Input dataset identifiers and relevant source object/table versions.

OpenLineage run ID and upstream/downstream lineage edges.

Output Iceberg table identifier and snapshot ID.

Great Expectations suite/checkpoint version and validation result.

DataHub metadata snapshot/record of ownership, classification and applicable assertions.

Identity/policy decisions for publication and privileged changes.

OpenTelemetry trace/run correlation identifiers and operational SLO status.

Retention/lifecycle policy applied to the product and associated evidence.

Exception records, if any, including owner, justification, compensating control and expiry date.